

TED UNIVERSITY

Student Management System Basis Path Test Report

SENG 311

Assignment: Homework 2: Basis Path Testing

Submission date: 07.11.2026

İsmail Çetintaşlı – ID 10002624

Umut Saraç – ID 10565404892

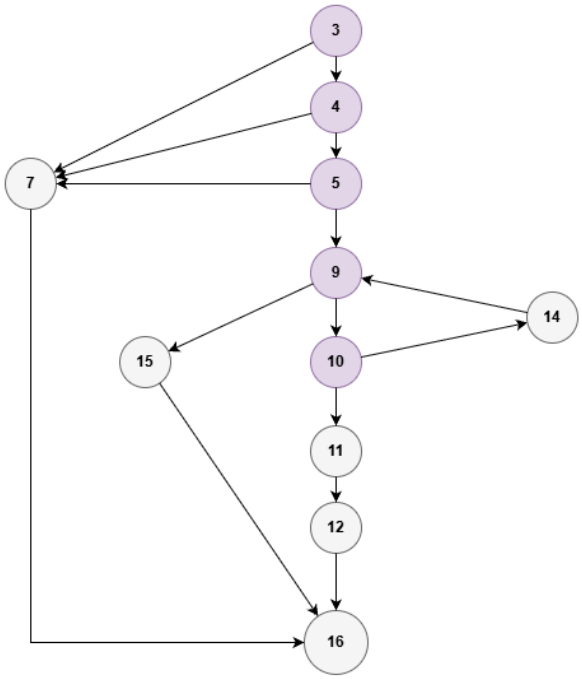
1. Project overview

As stated in the assignment, we each picked a method in the project that was featured in the previous assignment, constructed a flowchart in the style that was taught in the lectures and followed them for determining basis paths. We then wrote white box tests for each path.

We used PIT to subject some of the tests from HW1 to mutant testing (10 for each, a total of 20), the first being 33 - `updateStudent_existingPrn_replacesStudent` and the latter being 38 - `getAverageMarks_threeStudents_returnsAverage`.

2. Method Flowcharts

We picked `updateStudent` and `searchByName`, where the cyclomatic complexity of the former is 6 while the latter's is 5.



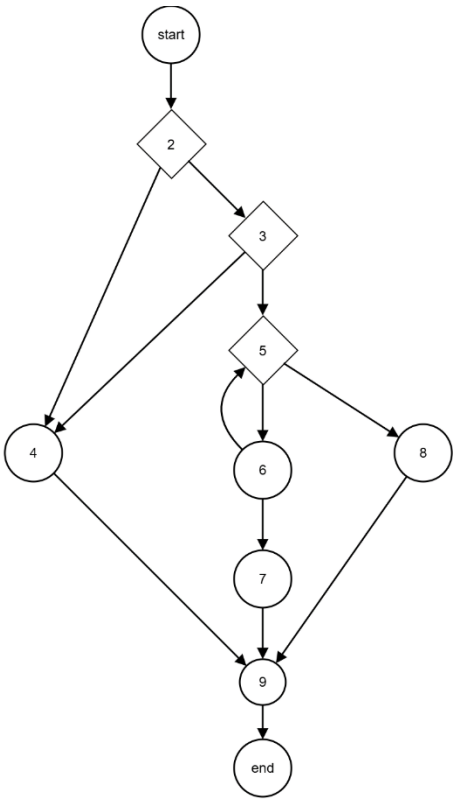

```
1 public void updateStudent(String prn, Student updatedStudent) {
2   try {
3     validatePRN(prn);
4     validatePRN(updatedStudent.getPrn());
5     validateMarks(updatedStudent.getMarks());
6   } catch (InvalidPRNException | InvalidMarksException e) {
7     throw new InvalidUpdateDataException(e.getMessage());
8   }
9   for (int i = 0; i < students.size(); i++) {
10    if (students.get(i).getPrn().equals(prn)) {
11      students.set(i, updatedStudent);
12      return;
13    }
14  } // i++ then back to 9
15  throw new StudentUpdateException("... not found");
16 }
```

Decision node (3, 4, 5, 9, 10)

Other node

N = 11, E = 15
V(G) = E - N + 2 = 15 - 11 + 2 = 6
Decisions + 1 = 5 + 1 = 6

```
1  Student searchByName(String name) {
2    if (name == null ||
3          name.trim().isEmpty())
4      throw new InvalidSearchCriteriaException(...);
5    for (Student student : students)
6      if (student.getName().equalsIgnoreCase(name))
7        return student;
8    throw new StudentNotFoundException(...);
9  }
```



3. White-box testing approach

Given that a path is related to the cyclomatic complexity directly, we have 6 and 5 basis paths to handle for the above methods, respectively. One thing to note is that the structure of the methods isn't merely made up of if-statements, and both contain for-loops. In this case, or any case where a loop is formed and a path looks like it contains another, a basis path can still be differentiated by actually tracking which nodes are visited by the program. This lets us cover all edges in determining the paths. Enumerating in this fashion:

`updateStudent`

- `(3)-(7)-(16)`, (PRN is invalid, an error is thrown)
- `(3)-(4)-(7)-(16)`, (PRN of the updated student is invalid, an error is thrown)
- `(3)-(4)-(5)-(7)-(16)`, (the marks of the student is invalid, an error is thrown)
- `(3)-(4)-(5)-(9)-(15)-(16)`, (the list is empty, thus the loop is unentered)
- `(3)-(4)-(5)-(9)-(10)-(11)-(12)-(16)`, (first one is a match, didn't loop)
- `(3)-(4)-(5)-(9)-(10)-(14)-(9)-(10)-(11)-(12)-(16)`, (second matched, loop made)

Following the same logic, here's the next method:

`searchByName`

- `(2)-(4)-(9)`, (name is null, relevant error was thrown)
- `(2)-(3)-(4)-(9)`, (name is found to be empty or whitespace, error thrown)
- `(2)-(3)-(5)-(8)-(9)`, (the list is empty, thus the loop was unentered and exited to an error)
- `(2)-(3)-(5)-(6)-(7)-(9)`, (first one is a match, didn't loop)
- `(2)-(3)-(5)-(6)-(5)-(6)-(7)-(9)`, (second matched, loop made)

4. Test inventory

We have created 18 test methods in total for these cases. In each basis-path class, the test for path N follows the Nth path listed in Section 3.

Test class	Tests	What they cover
`SearchByNameTest`	5	The 5 basis paths of searchByName
`UpdateStudentBasisPathTest`	7	The 6 basis paths of updateStudent, plus one extra test where a non-empty list has no match
`UpdateStudentDecisionTableTest`	6	Rules R1 to R6 of the updateStudent decision table (Section 7)

5. Mutation testing

We used PIT 1.18.2 along with its corresponding JUnit 5 plugin. PIT modifies the compiled bytecode, applying one change per mutant, and executes the selected test for each of these mutants. A mutant is considered "killed" if the test fails; it "survives" if the test continues to pass; and it has "no coverage" if the test does not reach the modified line of code at all.

For each run, we restricted the scope to a single HW1 test annotated with `@Tag("pit")` and excluded the other methods within the `StudentOperations` class, such that each run generated exactly 10 mutants for the specific method under test. The tables display the modified code. The mutated classes were decompiled using CFR; the complete mutated methods can be found in the files `pit_mutants_updateStudent.txt` and `pit_mutants_getAverageMarks.txt`, which have been submitted alongside the code.

Test 33 (`updateStudent_existingPrn_replacesStudent`)

#	Line	Original	Mutant	Result
0	112	`validatePRN(updatedStudent.getPrn())`	`validatePRN(null)`	Killed
1	113	`validateMarks(updatedStudent.getMarks())`	`validateMarks(0.0)`	Survived
2	115	`new InvalidUpdateDataException(e.getMessage())`	`new InvalidUpdateDataException(null)`	No coverage
3	118	`i < students.size()`	`i < 0`	Killed
4	118	`i < students.size()`	`i >= students.size()`	Killed
5	119	`students.get(i).getPrn().equals(prn)`	`false`	Killed
6	119	`students.get(i)`	`null (NullPointerException on .getPrn())`	Killed
7	119	`if (students.get(i).getPrn().equals(prn))`	`if (!students.get(i).getPrn().equals(prn))`	Survived
8	119	`students.get(i).getPrn()`	`null (NullPointerException on .equals())`	Killed
9	120	`students.set(i, updatedStudent)`	`call removed`	Killed

Test 33 replaces the second of three students with a dataset containing valid information and subsequently searches for this student using their PRN. During the grade verification process, Mutant 1 passes a value of 0.0 instead of the actual grades. Since both values are valid, the program's behavior remains unchanged. This scenario is covered by Test 35 from Homework 1 (HW1), which employs invalid grades. Mutant 7 inverts the PRN comparison, causing the first student to be replaced instead of the second. The modified dataset is still successfully located via its PRN, and Test 33 does not verify the first student or the specific positions. This case is addressed by Test 61 from Homework 1 (HW1), which checks the position. Mutant 2 alters the catch block, a section of code that is never reached by Test 33, as the data used are valid.

Test 38 (`getAverageMarks_threeStudents_returnsAverage`)

#	Line	Original	Mutant	Result
0	143	`if (students.isEmpty())`	`if (!students.isEmpty())`	Killed
1	143	`if (students.isEmpty())`	`if (false)`	Survived
2	143	`if (students.isEmpty())`	`check removed, isEmpty() result ignored`	Survived
3	143	`if (students.isEmpty())`	`if (true), so it always throws`	Killed
4	147	`double sum = 0`	`double sum = 1`	Killed
5	149	`sum += student.getMarks()`	`sum -= student.getMarks()`	Killed
6	149	`sum += student.getMarks()`	`sum += 0.0`	Killed
7	151	`sum / students.size()`	`sum * students.size()`	Killed
8	151	`return sum / students.size()`	`return 0.0`	Killed
9	151	`sum / students.size()`	`sum / 0.0`	Killed

Mutants 1 and 2 remove the check for an empty list. Since Test 38 always involves three students, the condition is never met, and the mutants behave just like the original code. Test 67 from Homework 1, which calls the method with an empty list, covers this case.

6. Execution results

Tests based on basis paths and decision tables: 18 runs, 17 passed and 1 failed. The failure concerns R6 which is explained in Section 7. All 12 tests in the two basis-path classes were successful.

```
> mvn test -Dtest=SearchByNameTest,UpdateStudentBasisPathTest,UpdateStudentDecisionTableTest

[INFO] Running SearchByNameTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0 -- in SearchByNameTest
[INFO] Running UpdateStudentBasisPathTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0 -- in UpdateStudentBasisPathTest
[INFO] Running UpdateStudentDecisionTableTest
[ERROR] Tests run: 6, Failures: 1, Errors: 0, Skipped: 0 <<< FAILURE! -- in
UpdateStudentDecisionTableTest
[ERROR] UpdateStudentDecisionTableTest.rule6_newPrnBelongsToAnother <<< FAILURE!
[ERROR] Tests run: 18, Failures: 1, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Mutation testing of test 33 (`updateStudent`):

```
> mvn -Ppit-update test-compile org.pitest:pitest-maven:mutationCoverage

>> Line Coverage (for mutated classes only): 29/79 (37%)
>> Generated 10 mutations Killed 7 (70%)
>> Mutations with no coverage 1. Test strength 78%
>> Ran 9 tests (0.9 tests per mutation)
[INFO] BUILD SUCCESS
```

Mutation testing of test 38 (`getAverageMarks`):

```
> mvn test-compile org.pitest:pitest-maven:mutationCoverage

>> Line Coverage (for mutated classes only): 25/79 (32%)
>> Generated 10 mutations Killed 8 (80%)
>> Mutations with no coverage 0. Test strength 80%
>> Ran 10 tests (1 tests per mutation)
[INFO] BUILD SUCCESS
```

7. Decision Tables

We picked updateStudent because its outcome depends on several checks made in a fixed order. C1 to C5 are the conditions and A1 to A4 are the actions.

Method: `updateStudent`

	R1	**R2**	**R3**	**R4**	**R5**	**R6**
C1 PRN searched for is valid	N	Y	Y	Y	Y	Y
C2 new student's PRN valid	–	N	Y	Y	Y	Y
C3 new marks in 0–100	–	–	N	Y	Y	Y
C4 PRN searched for is in list	–	–	–	N	Y	Y
C5 new PRN belongs to *another* student	–	–	–	–	N	Y
A1 `InvalidUpdateDataException`	X	X	X			
A2 `StudentUpdateException` (not found)				X		
A3 student replaced in place					X	
A4 `StudentUpdateException` (PRN already exists)						X
Previously Existing Test:	64	62	35	34	33	60

For every rule, there is a corresponding test in `UpdateStudentDecisionTableTest` (Section 8). The final row of the table refers to a test from HW1 that already verifies the same rule.

R6 fails, which is an expected outcome. Since `updateStudent` does not include a check for duplicates, the existing student entry is replaced rather than triggering an exception. Test 60 from HW1 had already uncovered this same issue, therefore, we have retained the test and left the code unchanged.

8. Test code

SearchByNameTest.java

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.Tag;

import static org.junit.jupiter.api.Assertions.*;

public class SearchByNameTest {
    private StudentOperations operations;
    private Student student1;

    @BeforeEach
    void setUp() throws Exception {
        operations = new StudentOperations();

        student1 = new Student("11111111111", "Ismail", "01.01.2006", 80.0);

        operations.addStudent(student1);
    }

    @Test
    @Tag("pit")
    @DisplayName("Test for path 1")
    void searchByName_path1() {
        assertThrows(StudentExceptions.InvalidSearchCriteriaException.class,
            () -> operations.searchByName(null)
        );
    }

    @Test
    @Tag("pit")
    @DisplayName("Test for path 2")
    void searchByName_path2() {
        assertThrows(StudentExceptions.InvalidSearchCriteriaException.class,
            () -> operations.searchByName("")
        );
    }

    @Test
    @Tag("pit")
    @DisplayName("Test for path 3")
    void searchByName_path3() {
        operations = new StudentOperations();

        assertThrows(StudentExceptions.StudentNotFoundException.class,
            () -> operations.searchByName("Ismail")
        );
    }

    @Test
    @Tag("pit")
    @DisplayName("Test for path 4")
    void searchByName_path4() throws Exception {
        assertSame(student1, operations.searchByName("Ismail"));
    }

    @Test
    @Tag("pit")
    @DisplayName("Test for path 5")
    void searchByName_path5() throws Exception {
```

```
        Student student2 = new Student("22222222222", "Eren", "01.01.2008", 85.0);
        operations.addStudent(student2);

        assertSame(student2, operations.searchByName("Eren"));
    }
}
```

UpdateStudentBasisPathTest.java

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

public class UpdateStudentBasisPathTest {

    private StudentOperations operations;
    private Student student1;
    private Student student2;

    @BeforeEach
    void setUp() throws Exception {
        operations = new StudentOperations();
        student1 = new Student("11111111111", "Ismail", "01.01.2006", 80.0);
        student2 = new Student("22222222222", "Eren", "01.01.2008", 85.0);
        operations.addStudent(student1);
        operations.addStudent(student2);
    }

    // P1: 3-7-16 (prn parametresi geçersiz)
    @Test
    void path1_invalidPrnParameter_throwsInvalidUpdateData() {
        Student updated = new Student("11111111111", "Ismail", "01.01.2006", 90.0);

        assertThrows(StudentExceptions.InvalidUpdateDataException.class,
                () -> operations.updateStudent("123", updated));
    }

    // P2: 3-4-7-16 (updatedStudent'in prn'i geçersiz)
    @Test
    void path2_invalidUpdatedStudentPrn_throwsInvalidUpdateData() {
        Student updated = new Student("123", "Ismail", "01.01.2006", 90.0);

        assertThrows(StudentExceptions.InvalidUpdateDataException.class,
                () -> operations.updateStudent("11111111111", updated));
    }

    // P3: 3-4-5-7-16 (marks geçersiz)
    @Test
    void path3_invalidMarks_throwsInvalidUpdateData() throws Exception {
        Student updated = new Student("11111111111", "Ismail", "01.01.2006", 150.0);

        StudentExceptions.InvalidUpdateDataException ex = assertThrows(
                StudentExceptions.InvalidUpdateDataException.class,
                () -> operations.updateStudent("11111111111", updated));

        assertTrue(ex.getMessage().startsWith("Invalid update data:"));
        assertSame(student1, operations.searchByPrn("11111111111"));
    }

    // P4: 3-4-5-9-15-16 (liste boş, döngüye girilmez, bulunamadı)
    @Test
    void path4_emptyList_throwsStudentUpdateException() {
```

```
        StudentOperations emptyOps = new StudentOperations();
        Student updated = new Student("11111111111", "Ismail", "01.01.2006", 90.0);

        StudentExceptions.StudentUpdateException ex = assertThrows(
                StudentExceptions.StudentUpdateException.class,
                () -> emptyOps.updateStudent("11111111111", updated));

        assertEquals("Failed to update student: Student with PRN 11111111111 not found",
                ex.getMessage());
    }

    // P5: 3-4-5-9-10-11-12-16 (ilk elemanda bulundu)
    @Test
    void path5_foundAtFirstElement_replacesStudent() throws Exception {
        Student updated = new Student("11111111111", "Ismail Updated", "01.01.2006", 95.0);

        operations.updateStudent("11111111111", updated);

        assertSame(updated, operations.searchByPrn("11111111111"));
        assertSame(student2, operations.searchByPrn("22222222222"));
    }

    // P6: 3-4-5-9-10-14-9-10-11-12-16 (ilkinde eşleşmedi, ikincide bulundu)
    @Test
    void path6_foundAtSecondElement_replacesStudent() throws Exception {
        Student updated = new Student("22222222222", "Eren Updated", "01.01.2008", 95.0);

        operations.updateStudent("22222222222", updated);

        assertSame(updated, operations.searchByPrn("22222222222"));
        assertSame(student1, operations.searchByPrn("11111111111"));
    }

    // Ekstra (basis path değil): 3-4-5-9-10-14-9-10-14-9-15-16
    // Dolu listede hiçbir eleman eşleşmez, döngü biter, 15'e gidilir
    @Test
    void extra_nonEmptyListNotFound_throwsStudentUpdateException() {
        Student updated = new Student("444444444444", "Nobody", "01.01.2000", 50.0);

        assertThrows(StudentExceptions.StudentUpdateException.class,
                () -> operations.updateStudent("444444444444", updated));
    }
}
```

UpdateStudentDecisionTableTest.java

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

// One test per rule (R1-R6) of the updateStudent decision table in the report.
public class UpdateStudentDecisionTableTest {
    private StudentOperations operations;
    private Student student1;
    private Student student2;

    @BeforeEach
    void setUp() throws Exception {
        operations = new StudentOperations();

        student1 = new Student("11111111111", "Ismail", "01.01.2006", 80.0);
```

```
        student2 = new Student("22222222222", "Eren", "01.01.2008", 85.0);

        operations.addStudent(student1);
        operations.addStudent(student2);
    }

    @Test
    @DisplayName("R1: searched PRN is invalid -> InvalidUpdateDataException")
    void rule1_searchedPrnInvalid() throws Exception {
        Student updated = new Student("11111111111", "New", "01.01.2006", 90.0);

        assertThrows(StudentExceptions.InvalidUpdateDataException.class,
                () -> operations.updateStudent("123", updated));
        assertSame(student1, operations.searchByPrn("11111111111"));
    }

    @Test
    @DisplayName("R2: new student's PRN is invalid -> InvalidUpdateDataException")
    void rule2_newPrnInvalid() throws Exception {
        Student updated = new Student("456", "New", "01.01.2006", 90.0);

        assertThrows(StudentExceptions.InvalidUpdateDataException.class,
                () -> operations.updateStudent("11111111111", updated));
        assertSame(student1, operations.searchByPrn("11111111111"));
    }

    @Test
    @DisplayName("R3: new marks are outside 0-100 -> InvalidUpdateDataException")
    void rule3_marksInvalid() throws Exception {
        Student updated = new Student("11111111111", "New", "01.01.2006", 150.0);

        assertThrows(StudentExceptions.InvalidUpdateDataException.class,
                () -> operations.updateStudent("11111111111", updated));
        assertSame(student1, operations.searchByPrn("11111111111"));
    }

    @Test
    @DisplayName("R4: searched PRN is not in the list -> StudentUpdateException (not found)")
    void rule4_searchedPrnNotInList() {
        Student updated = new Student("44444444444", "New", "01.01.2006", 90.0);

        StudentExceptions.StudentUpdateException exception = assertThrows(
                StudentExceptions.StudentUpdateException.class,
                () -> operations.updateStudent("44444444444", updated));

        assertTrue(exception.getMessage().contains("not found"));
    }

    @Test
    @DisplayName("R5: all valid, new PRN free -> student replaced in place")
    void rule5_validUpdate_replacesInPlace() throws Exception {
        Student updated = new Student("11111111111", "New", "01.01.2006", 90.0);

        operations.updateStudent("11111111111", updated);

        assertSame(updated, operations.searchByPosition(0));
    }

    // The existing test failed on this part last homework
    @Test
    @DisplayName("R6: new PRN belongs to another student -> StudentUpdateException (already
exists)")
    void rule6_newPrnBelongsToAnother() {
        Student updated = new Student("22222222222", "New", "01.01.2006", 90.0);
```

```
        StudentExceptions.StudentUpdateException exception = assertThrows(
                StudentExceptions.StudentUpdateException.class,
                () -> operations.updateStudent("11111111111", updated));

        assertTrue(exception.getMessage().contains("already exists"));
    }
}
```

HW1 tests used for mutation testing

```
// 33
@Test
@Tag("pit")
void updateStudent_existingPrn_replacesStudent() throws Exception {
    Student updated = new Student("22222222222", "Eren Updated", "01.01.2008", 95.0);

    operations.updateStudent("22222222222", updated);

    Student found = operations.searchByPrn("22222222222");
    assertAll("updated student",
            () -> assertNotNull(found),
            () -> assertSame(updated, found),
            () -> assertNotSame(student2, found),
            () -> assertEquals("Eren Updated", found.getName()),
            () -> assertEquals(95.0, found.getMarks(), 0.001));
}


// 38
@Test
@Tag("pit")
void getAverageMarks_threeStudents_returnsAverage() throws Exception {
    assertEquals(85.0, operations.getAverageMarks(), 0.001);
}
```